

Spark - Guia completa de arquitectura y optimizacion avanzada

Manual tecnico en espanol - Arquitectura, ejecucion y rendimiento

spark-ui-performance-lab

2026

Spark - Guia completa de arquitectura y optimizacion avanzada

Este documento es una referencia complementaria en espanol para reforzar los fundamentos de Spark que aparecen en el lab.

El runtime del proyecto sigue siendo:

- Apache Spark Standalone.
- Scala.
- Docker Compose.
- Spark History Server.
- Redpanda opcional para streaming.

Los apartados sobre Databricks, Delta Lake, Photon o PySpark se incluyen como contexto de produccion, no como dependencias del lab.

Indice

1. Arquitectura de Spark
2. Ciclo de vida de una query
3. Modelo de computacion distribuida
4. Data skew
5. Shuffle, joins y broadcast
6. Particiones, paralelismo y memoria
7. PySpark en produccion
8. Delta Lake
9. Photon, JVM y GC
10. Autoscaling vs single node
11. AQE
12. Playbooks y checklists
13. Chuleta Spark Master

1. Arquitectura de Spark

Spark se basa en una arquitectura distribuida con un proceso **Driver** que coordina la ejecucion, un **Cluster Manager** que asigna recursos y varios **Workers** que ejecutan procesos **Executor**.

Cada executor ejecuta **tasks**. Cada task procesa normalmente una particion de datos.

Componente	Rol
Driver	Coordina la ejecucion, construye planes logicos/fisicos y envia tasks a los executors.
Cluster Manager	Asigna CPU y memoria. Puede ser Standalone, YARN, Kubernetes o un entorno gestionado.
Worker Node	Servidor, VM o contenedor que aloja uno o varios executors.
Executor	Proceso JVM que ejecuta tasks y mantiene cache local.
Task	Unidad minima de ejecucion. Normalmente procesa una particion.
Stage	Conjunto de tasks sin dependencia de shuffle entre ellas.
Job	Conjunto de stages disparado por una accion como count, write, collect o show.

Flujo conceptual:

```
df1 = spark.read.csv("/mnt/landing/fact.csv", header=True)
df2 = spark.read.csv("/mnt/landing/dim.csv", header=True)

res = df1.join(df2, "id").filter("country = 'ES'")

res.write.mode("overwrite").parquet("/mnt/curated/out")
```

En Spark UI, este flujo aparece repartido entre:

- **Jobs:** acciones ejecutadas.
- **Stages:** fases separadas por shuffle.
- **SQL/DataFrame:** plan logico, plan fisico y operadores.
- **Executors:** uso de recursos, tasks, memoria, shuffle y GC.

2. Ciclo de vida de una query

El ciclo de vida tipico de una query es:

```
DataFrame / SQL
-> Logical Plan
-> Optimized Logical Plan
-> Physical Plan
-> Execution
-> Runtime metrics
```

Spark usa Catalyst para optimizar el plan logico y Tungsten para optimizar ejecucion JVM, memoria y generacion de codigo.

Ejemplo SQL:

```
EXPLAIN FORMATTED
SELECT /*+ BROADCAST(d) */ f.*
FROM fact f
JOIN dim d ON f.k = d.k
WHERE f.dt >= DATE '2025-10-01';
```

En Spark UI, usa:

- **Plan Visualization** para entender la forma del plan.
- **Plan Details** para buscar operadores concretos como Exchange, SortMergeJoin, BroadcastHashJoin o AdaptiveSparkPlan.

3. Modelo de computacion distribuida

Spark divide los datasets en particiones. Cada particion se procesa mediante una task en un executor.

Transformaciones **narrow**:

- No requieren mover datos entre executors.
- Ejemplos: select, filter, columnas calculadas simples.

Transformaciones **wide**:

- Requieren redistribuir datos.
- Crean shuffle.
- Ejemplos: join, groupBy, distinct, orderBy, repartition.

Regla practica:

- Objetivo orientativo de archivo: 128-512 MB.
- Shuffle partitions: empezar cerca de cores totales x 2-3 y ajustar con evidencia.

4. Data Skew

Data skew es un desbalance extremo de datos por clave o particion. El job termina cuando termina la task mas lenta, por eso una sola particion muy grande puede dominar toda la ejecucion.

Sintomas en Spark UI:

- Una task tarda mucho mas que el resto.
- Max muy superior a Median o 75th percentile.
- Stages con stragglers.
- Shuffle o input muy concentrado en pocas tasks.

Consulta para detectar claves calientes:

```
SELECT customer_id, COUNT(*) AS c
FROM t
GROUP BY customer_id
ORDER BY c DESC
LIMIT 10;
```

En PySpark:

```
import pyspark.sql.functions as F

df.groupBy("customer_id") \
    .count() \
    .orderBy(F.desc("count")) \
    .show(10)
```

Soluciones habituales:

1. Activar AQE y skew join handling.
2. Reparticionar por la clave adecuada.
3. Aplicar salting en claves calientes.
4. Pre-agregar antes de joins grandes.
5. Separar outliers en un flujo específico.
6. Revisar particionamiento físico de datos.

Configuración orientativa:

```
SET spark.sql.adaptive.enabled = true;
SET spark.sql.adaptive.skewJoin.enabled = true;
SET spark.sql.adaptive.skewedPartitionThresholdInBytes = 268435456;
```

5. Shuffle, Joins y Broadcast

Shuffle es una de las operaciones más caras en Spark porque mueve datos entre executors.

Se produce habitualmente en:

- join
- groupBy
- distinct
- orderBy
- repartition

Evidencia en Spark UI:

- Exchange en el plan físico.
- Nuevos límites de stage.
- Shuffle read/write en Stages y Executors.

Broadcast join evita shufflear el lado pequeño de un join.

Configuración y uso:

```
SET spark.sql.autoBroadcastJoinThreshold = 200MB;
SET spark.sql.broadcastTimeout = 600;
```

```
SELECT /*+ BROADCAST(dim) */ *
FROM fact f
JOIN dim ON f.k = dim.k;
```

En PySpark:

```
import pyspark.sql.functions as F

res = fact.join(F.broadcast(dim), "customer_id")
```

En Spark UI:

- SortMergeJoin suele indicar join con shuffle.
- BroadcastHashJoin y BroadcastExchange indican broadcast.

6. Particiones, paralelismo y memoria

Las particiones controlan cuantas tasks puede generar Spark.

Demasiadas pocas particiones:

- Pocas tasks.
- Cores o executors infrautilizados.
- Jobs aparentemente lentos por falta de paralelismo.

Demasiadas particiones:

- Muchas tasks pequenas.
- Overhead de planificacion.
- Scheduler delay mas visible.

Configuracion comun:

```
SET spark.sql.shuffle.partitions = 192;  
SET spark.sql.files.maxPartitionBytes = 134217728;
```

Reglas rapidas:

- Usa repartition(n, col) para redistribuir con shuffle y balancear joins/aggregations.
- Usa coalesce(n) para reducir particiones evitando shuffle cuando sea razonable.
- Ajusta siempre mirando Spark UI, no solo formulas.

7. PySpark en produccion

Evita UDFs Python cuando existan funciones nativas.

Menos recomendable:

```
from pyspark.sql.functions import udf  
  
@udf("double")  
def my_udf(x):  
    return x * 1.21  
  
df = df.withColumn("price_vat", my_udf("price"))
```

Mejor:

```
import pyspark.sql.functions as F  
  
df = df.withColumn("price_vat", F.col("price") * F.lit(1.21))
```

Persistencia:

```
from pyspark import StorageLevel  
  
df_small = (  
    df  
    .filter("dt >= '2025-01-01'")  
    .select("k", "v")  
    .persist(StorageLevel.MEMORY_AND_DISK_SER)  
)  
  
_ = df_small.count()
```

Materializa el cache con una accion y libera memoria con unpersist() cuando deje de ser necesario.

8. Delta Lake

Delta Lake no forma parte del runtime de este lab, pero es habitual en produccion.

Operaciones utiles:

```
OPTIMIZE sales;  
OPTIMIZE sales ZORDER BY (customer_id, order_date);  
VACUUM sales RETAIN 168 HOURS;  
ANALYZE TABLE sales COMPUTE STATISTICS;
```

Cuando usar Z-ORDER:

- Filtros frecuentes por columnas de alta cardinalidad.
- Queries analiticas con WHERE sobre esas columnas.
- No suele aportar en full scans o tablas temporales pequenas.

9. Photon, JVM y GC

Photon es especifico de Databricks. Ejecuta partes del motor en C++ y reduce el impacto de la JVM/GC en ciertos workloads.

En Spark JVM clasico, las senales de memoria se leen en:

- Stage detail: spill, peak execution memory, GC time.
- Executors: Task Time (GC Time), storage memory, failed tasks.

Ejemplo de configuracion Databricks:

```
spark.databricks.photon.enabled = true  
spark.executor.cores = 5  
spark.executor.memory = 8g  
spark.executor.memoryOverhead = 2g
```

10. Autoscaling vs Single Node

Autoscaling puede ayudar cuando:

- Los jobs duran muchos minutos u horas.
- El workload cambia de tamano.
- El coste de levantar nodos compensa la duracion del job.

Puede no ayudar cuando:

- Jobs muy cortos.
- Overhead de escalado mayor que el beneficio.
- Pruebas pequenas o compactaciones controladas.

Single Node puede ser adecuado para:

- Pruebas.
- Jobs pequenos.

- Compactaciones o mantenimiento controlado.

11. AQE

AQE permite que Spark adapte el plan físico durante la ejecución usando estadísticas reales.

Puede:

- Coalescer particiones de shuffle.
- Manejar skew.
- Cambiar estrategia de join.

Configuración habitual:

```
SET spark.sql.adaptive.enabled = true;  
SET spark.sql.adaptive.coalescePartitions.enabled = true;  
SET spark.sql.adaptive.skewJoin.enabled = true;  
SET spark.sql.adaptive.shuffle.targetPostShuffleInputSize = 134217728;
```

Evidencia en Spark UI:

- AdaptiveSparkPlan
- AQEShuffleRead
- Diferencia entre initial plan y final plan.

12. Playbooks y Checklists

Job lento

- Revisar SQL/DataFrame plan.
- Buscar Exchange, tipo de join y ordenaciones.
- Revisar Spark UI para skew, spill y outliers.
- Ajustar AQE y shuffle partitions.
- Usar broadcast si un lado es pequeño.
- Aplicar filtros/proyecciones temprano.
- Reparticionar por clave relevante.
- Si sigue mal: salting y pre-agregaciones.

OOM o GC alto

- Reducir tamaño de partición aumentando particiones de shuffle.
- Usar niveles de persistencia con disco si procede.
- Evitar UDFs Python si hay funciones nativas.
- Seleccionar solo columnas necesarias.
- Revisar Task Time (GC Time) en Executors.
- Revisar spill en Stage detail.

Muchos archivos pequeños

- Reescribir con un número razonable de particiones.
- Controlar maxRecordsPerFile.
- Compactar si el formato/lakehouse lo permite.
- Validar en Spark UI si bajan las tasks pequeñas.

13. Chuleta Spark Master

AQE y skew

```
SET spark.sql.adaptive.enabled = true;
SET spark.sql.adaptive.coalescePartitions.enabled = true;
SET spark.sql.adaptive.skewJoin.enabled = true;
```

Shuffle y files

```
SET spark.sql.shuffle.partitions = 192;
SET spark.sql.files.maxPartitionBytes = 134217728;
```

Broadcast

```
SET spark.sql.autoBroadcastJoinThreshold = 200MB;
SET spark.sql.broadcastTimeout = 600;
```

Arrow para PySpark vectorizado

```
SET spark.sql.execution.arrow.pyspark.enabled = true;
```

Patron PySpark comun

```
df = (
    spark.read.format("delta")
    .load("/mnt/bronze/fact_sales")
    .select("k", "ts", "amount")
    .filter("ts >= '2025-09-01'")
)

df = df.repartition(192, "k")
res = df.join(F.broadcast(dim), "k")
```

Escritura con control de archivos

```
(
    df.repartition(256)
    .write
    .option("maxRecordsPerFile", 2_000_000)
    .format("delta")
    .mode("overwrite")
    .save("/mnt/gold/sales_by_customer")
)
```